

Lab - Azure Blobs - Optimistic Concurrency

To Begin with the Lab

Summary

This lab demonstrates how to implement **Optimistic Concurrency** in Azure Blob Storage using **ETags**. The application retrieves the blob's current ETag before making changes and only updates the blob if the ETag has not changed. If another user modifies the blob first, Azure prevents the update by returning an error.

Understanding

Optimistic Concurrency is a technique used to prevent accidental overwriting of data when multiple users or applications access the same blob simultaneously.

Every Azure Blob has an **ETag (Entity Tag)**, which acts as a unique version identifier.

The process works as follows:

1. Read the blob.
2. Retrieve its current ETag.
3. Make changes in memory.
4. Upload the blob only if the ETag still matches.
5. If someone modified the blob in the meantime, the ETag changes and Azure rejects the update.

This ensures that only one version of the blob is updated, preventing data loss.

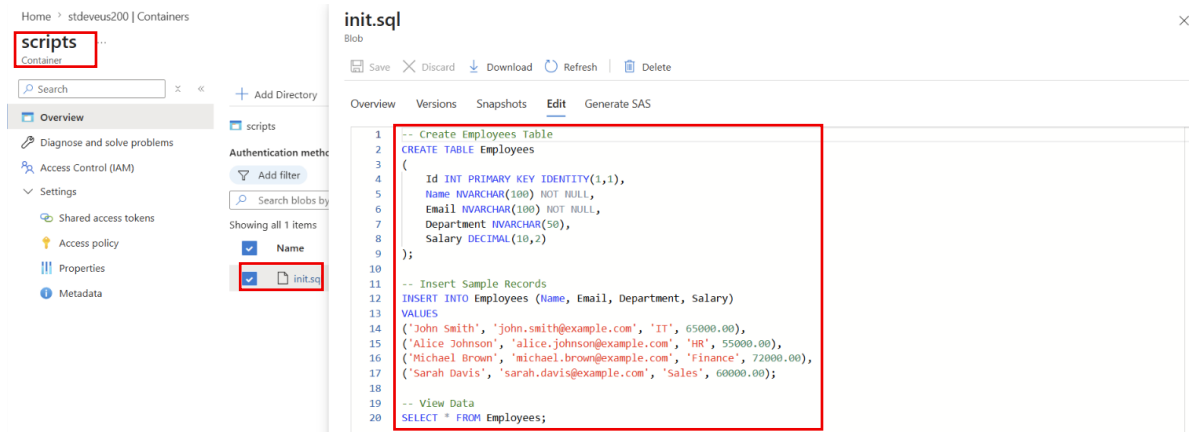
Example

- User A reads **init.sql** (ETag = ABC123).
 - User B edits and saves the blob.
 - Azure changes the ETag to XYZ789.
 - User A tries to upload using the old ETag (ABC123).
 - Azure detects the mismatch and throws an exception.
 - User A must download the latest blob before trying again.
-

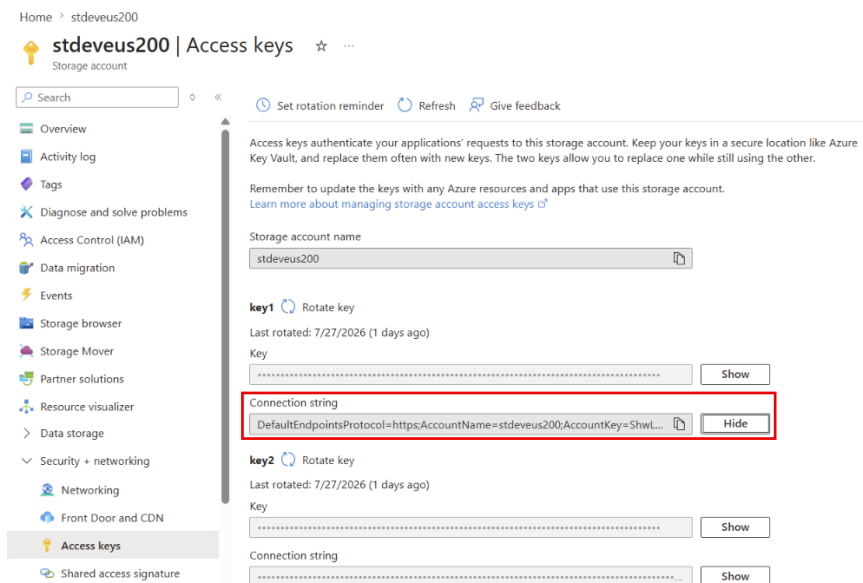
Lab Steps

Step 1: Create the Azure Storage Resources

- Create an Azure Storage Account.
- Create a Blob Container named **scripts**.
- Upload **init.sql** to the container.
- Verify that the blob contains sample SQL data.

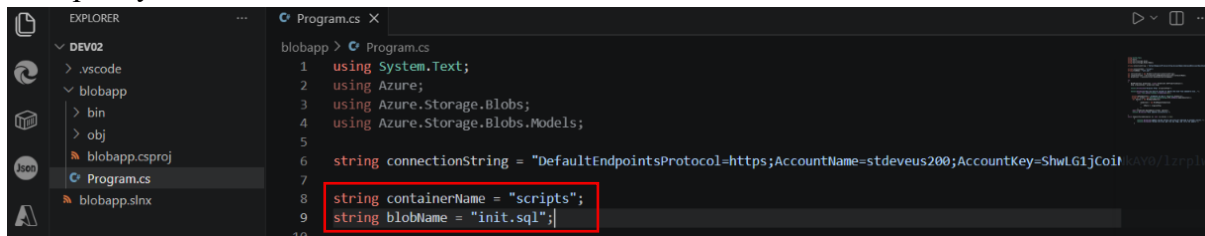


- Go to the **Azure Storage Accounts**.
- Copy the **Connection string**.
- Add the Azure Storage **connection string**.



Step 2: Create the .NET Console Application

- Create a Console Application.
- Install the **Azure.Storage.Blobs** NuGet package.
- Add the Azure Storage **connection string**.
- Specify the **container name** and **blob name**.



Step 3: Create Blob Clients

- Create a BlobServiceClient.
- Get a reference to the Blob Container.
- Get a reference to the target Blob (init.sql).

```
Program.cs X
blobapp > Program.cs
1 using System.Text;
2 using Azure;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.Models;
5
6 string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdeveus200;AccountKey=ShwLG1jCoitKAY0/1zrplk";
7
8 string containerName = "scripts";
9 string blobName = "init.sql";
10
11 var serviceClient = new BlobServiceClient(connectionString);
12 var containerClient = serviceClient.GetBlobContainerClient(containerName);
13 var blobClient = containerClient.GetBlobClient(blobName);
14
```

Step 4: Retrieve Blob Properties

- Call `GetProperties()` on the blob.
- Read the blob's current **ETag**.
- Store the ETag for later validation.
- Display the ETag value on the console.

```
Program.cs X
blobapp > Program.cs
1 using System.Text;
2 using Azure;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.Models;
5
6 string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdeveus200;AccountKey=ShwLG1jCoitKAY0/1zrplk";
7
8 string containerName = "scripts";
9 string blobName = "init.sql";
10
11 var serviceClient = new BlobServiceClient(connectionString);
12 var containerClient = serviceClient.GetBlobContainerClient(containerName);
13 var blobClient = containerClient.GetBlobClient(blobName);
14
15 try
16 {
17     BlobProperties properties = await blobClient.GetPropertiesAsync();
18     ETag originalETag = properties.ETag;
19
20     Console.WriteLine($"Original ETag: {originalETag}");
21
22     Console.WriteLine("You now have 45 seconds to modify the blob from somewhere else...");
23     await Task.Delay(TimeSpan.FromSeconds(45));
24
25     string updatedContent = $"Updated by app at {DateTime.UtcNow:u}";
26     using var stream = new MemoryStream(Encoding.UTF8.GetBytes(updatedContent));
27     var options = new BlobUploadOptions
28     {
29         Conditions = new BlobRequestConditions
30         {
31             IfMatch = originalETag
32         }
33     };
34     await blobClient.UploadAsync(stream, options);
35     Console.WriteLine("Blob updated successfully.");
36 }
37
```

Step 5: Pause Before Updating

- Pause the application for approximately **45 seconds**.
- Use this time to manually edit the blob in Azure Portal or Azure Storage Explorer.
- Saving the blob generates a new ETag.

init.sql

Blob

Save Discard Download Refresh Delete

Saved

Overview Versions Snapshots Edit Generate SAS

```
1 Updated by app at 2026-07-28 07:52:10Z---
2
```

Step 6: Prepare Updated Content

- Create the new blob content.
- Store the content inside a MemoryStream.
- Prepare the data for upload.
- Run commands

```
dotnet build
```

```
dotnet run
```

```
powershell blobapp X
PS D:\Dev2\Dev02\blobapp> dotnet build
Restore complete (1.2s)
blobapp net10.0 succeeded (0.5s) -> bin\Debug\net10.0\blobapp.dll
Build succeeded in 3.2s
PS D:\Dev2\Dev02\blobapp> dotnet run
Original ETag: "0x8DEEC7D20ED4AD8"
```

Step 7: Configure Upload Conditions

- Create BlobUploadOptions.
- Set the request condition using the previously retrieved ETag.
- Specify **IfMatch = Existing ETag**.
- This ensures Azure uploads only if the blob has not changed.

```
PS D:\Dev2\Dev02\blobapp> dotnet build
Restore complete (1.2s)
blobapp net10.0 succeeded (0.5s) -> bin\Debug\net10.0\blobapp.dll
Build succeeded in 3.2s
PS D:\Dev2\Dev02\blobapp> dotnet run
Original ETag: "0x8DEEC7D20ED4AD8"
You now have 45 seconds to modify the blob from somewhere else...
Update failed because the blob was modified by another process.
Reload the blob, get the new ETag, and retry the update.
PS D:\Dev2\Dev02\blobapp>
```

Step 8: Upload the Blob

- Upload the updated content using the Blob Client.
- Pass the upload options containing the ETag condition.
- Azure compares the current ETag with the stored ETag.
- Current ETag.

init.sql
Blob

Save Discard Download Refresh Delete Change tier Acquire lease Break lease Give feedback

Overview Versions Snapshots Edit Generate SAS

Properties

URL	https://stdevus200.blo...
LAST MODIFIED	7/28/2026, 1:34:44 PM
CREATION TIME	7/28/2026, 11:25:23 AM
VERSION ID	-
TYPE	Block blob
SIZE	43 B
ACCESS TIER	Hot (Inferred)
ACCESS TIER LAST MODIFIED	N/A
ARCHIVE STATUS	-
REHYDRATE PRIORITY	-
SERVER ENCRYPTED	true
ETAG	0x8DEEC7EE26F37F4
VERSION-LEVEL IMMUTABILITY POLICY	Disabled
CACHE-CONTROL	
CONTENT-TYPE	application/octet-stream
CONTENT-MD5	AH0MkzOjmUfQ1twaXShCl...
CONTENT-ENCODING	
CONTENT-LANGUAGE	
CONTENT-DISPOSITION	

- Stored ETag.

```

powershell blobapp X
PS D:\Dev2\Dev02\blobapp> dotnet build
Restore complete (1.2s)
  blobapp net10.0 succeeded (0.5s) → bin\Debug\net10.0\blobapp.dll

Build succeeded in 3.2s
PS D:\Dev2\Dev02\blobapp> dotnet run
Original ETag: "0x8DEEC7D20ED4AD8"
You now have 45 seconds to modify the blob from somewhere else...
Update failed because the blob was modified by another process.
Reload the blob, get the new ETag, and retry the update.
PS D:\Dev2\Dev02\blobapp>

```

Step 9: Handle Concurrency Errors

- Wrap the upload operation in a try-catch block.
- Catch the exception generated when the ETag does not match.
- Display a message indicating that another user has modified the blob.

```

powershell blobapp X
PS D:\Dev2\Dev02\blobapp> dotnet build
Restore complete (1.2s)
  blobapp net10.0 succeeded (0.5s) → bin\Debug\net10.0\blobapp.dll

Build succeeded in 3.2s
PS D:\Dev2\Dev02\blobapp> dotnet run
Original ETag: "0x8DEEC7D20ED4AD8"
You now have 45 seconds to modify the blob from somewhere else...
Update failed because the blob was modified by another process.
Reload the blob, get the new ETag, and retry the update.
PS D:\Dev2\Dev02\blobapp>

```

Step 10: Verify the Result

- Run the application without modifying the blob.

- Observe that the upload succeeds.
- Run the application again and edit the blob during the waiting period.
- Observe that the upload fails because the ETag has changed.

```
powershell blobapp X

PS D:\Dev2\Dev02\blobapp> dotnet build
Restore complete (1.2s)
  blobapp net10.0 succeeded (0.5s) → bin\Debug\net10.0\blobapp.dll

Build succeeded in 3.2s
PS D:\Dev2\Dev02\blobapp> dotnet run
Original ETag: "0x8DEEC7D20ED4AD8"
You now have 45 seconds to modify the blob from somewhere else...
Update failed because the blob was modified by another process.
Reload the blob, get the new ETag, and retry the update.
PS D:\Dev2\Dev02\blobapp> dotnet build
Restore complete (1.2s)
  blobapp net10.0 succeeded (0.6s) → bin\Debug\net10.0\blobapp.dll

Build succeeded in 3.3s
PS D:\Dev2\Dev02\blobapp> dotnet run
Original ETag: "0x8DEEC7EE26F37F4"
You now have 45 seconds to modify the blob from somewhere else...
Blob updated successfully.
PS D:\Dev2\Dev02\blobapp>
```